GEMINI

Helix Constitution — Universal GEMINI.md

Field	Value
Revision	8
Created	2026-06-09
Last modified	2026-07-07T19:05:34Z
Status	active

Status summary	Added §11.4.184 — Mandatory SonarQube static-analysis CLI + local tooling installed and PATH-discoverable (User mandate 2026-07-06): the SonarQube scanner CLI (`sonar-scanner`) MUST be installed AND durably PATH-discoverable via `.bashrc` / `.zshrc` (exactly like the Firebase/GitHub/GitLab CLIs), PLUS the shared `constitution/scripts/sonarqube/` tooling consumed BY REFERENCE (§11.4.28/§11.4.177/§11.4.80-style, NEVER copied); proven GREEN by `sonarqube_install_check.sh` exit 0 + `command -v sonar-scanner` (anti-bluff §11.4.6, never assumed); rootless podman §11.4.161, never rootful docker/sudo; DISTINCT from the repealed §11.4.166 Semgrep mandate — mandates TOOLING availability, not a scan on every build. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-184-PROPAGATION` + recommended gate `CM-SONARQUBE-CLI-INSTALLED` + paired §1.1 mutation. Landed in numerical order between §11.4.183 and the already-present §11.4.185 (union resolution — nothing dropped). CLAUDE.md + AGENTS.md + QWEN.md + GEMINI.md mirrors updated in lockstep (§11.4.157). Prior round: Added §11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate 2026-06-25): every change to ANY user-facing UI surface MUST be proven by DEVICE-INDEPENDENT host-side RENDERED PIXELS before claimed correct — the real component rendered to a PNG ON THE HOST (no device/emulator/running app; Compose→Roborazzi/Paparazzi, web→Playwright/Storybook, SwiftUI→snapshot-testing, equiv per stack) for EVERY screen×state×{light,dark} theme, dual-validated by (i) golden image-diff AND (ii) an OCR/vision oracle reading rendered text+labels+control bounds (NO overlap / label-over-label / clipping / off-screen / collapsed-or-giant-unbounded widget). VALUE/token-equality / property-assertion UI tests are FORBIDDEN as the PROOF a UI is correct (forensic FACT 2026-06-25: hex/sp/dp value-equality unit tests stayed GREEN while the operator opened a broken giant-button screen) — may supplement, NEVER substitute the rendered-pixel proof; self-validated golden-good/golden-bad analyzer §11.4.107(10); device-offline is NEVER a valid skip (host-render IS the device-independent path); COMPLEMENTS not replaces §11.4.153/.158/.159/.160 live-device recording + §11.4.162 OpenDesign tokens + §11.4.168 exported-doc visual validation. Propagation gate CM-COVENANT-114-170-PROPAGATION + recommended gate CM-HOST-RENDERED-UI-VISUAL-PROOF + paired §1.1 mutation. Classification: universal (§11.4.17). Prior round: §11.4.166 REPEALED (operator decision 2026-06-22): Universal Semgrep static-analysis mandate repealed — Semgrep no longer mandatory; scaffolding, submodules/semgrep, MCP/

Field	Value
	pre-commit/PATH wiring, docs_chain semgrep_status context, and CM-COVENANT-114-166-PROPAGATION/CM-SEMGREP-WIRED gates removed. Added §11.4.163 mirror — Universal Media Validation: every recorded artifact MUST pass MEDIA VALIDATION pipeline (OCR/transcription/text parsing vs patterns, self-validated analyzer, structured verdict, pinpoint on FAIL, real-time trigger, §1.1 mutation). Added §11.4.164 mirror — Universal Auto-Propagation Hook: post_update_hook.sh detects/registers/installs skills/MCP/hooks/scripts; consumer invokes after every pull. Added §11.4.165 mirror — Universal Independent Verification Agent: every output passes INDEPENDENT verifier (§11.4.70/.20), zero-finding GO. All Classification: universal (§11.4.17). Prior round: Added §11.4.160–162 mirror entries — Vision-verified recording + HelixQA bridge (User mandate 2026-06-21): every video recording MUST be processed through a vision/OCR pipeline reading on-screen content against SPECIFY-phase expected patterns; ≤5s frame capture, self-validated analyzer, per-frame PASS/FAIL with evidence path. Rootless container runtime (User mandate 2026-06-21): Podman rootless mandatory; rootful Docker/sudo FORBIDDEN unless §11.4.112; vasic-digital/containers sole orchestration layer. OpenDesign UI design system (User mandate 2026-06-21): mandatory UI design-and-refinement system for all user-facing interfaces; design tokens for color/typography/spacing; light+dark themes; extend upstream per §11.4.74. All three Classification: universal (§11.4.17). GEMINI.md mirrors updated in lockstep per §11.4.157. Prior round: New Gemini CLI carrier of the Helix Constitution — added by §11.4.140 (universal action-prefix system) so Gemini CLI gets the LAYER-1 action-prefix recognition floor out of the box, plus the §11.4.141 token-efficiency mandate mirror. This file is one of the four canonical agent context carriers (`CLAUDE.md` , `AGENTS.md` , `QWEN.md` , `GEMINI.md`) the constitution maintains per §11.4.35; the textual identity of the action-prefix recognition block across all four IS the LAYER-1 universality. This GEMINI.md is a thin carrier — the full universal rule set is in `Constitution.md` (canonical) and the sibling carriers; Gemini CLI reads this file as its context carrier. Back-filled §11.4.142–157 to lockstep (§11.4.157).
Issues	none

Field	Value
Issues summary	—
Fixed	§11.4.140 + §11.4.141 mirrors
Fixed summary	Carrier created in lockstep with the Constitution.md §11.4.140 + §11.4.141 additions.
Continuation	—

How inheritance works

This GEMINI.md is the **Gemini CLI carrier** of the Helix Constitution submodule. It is one of the four canonical agent context carriers (`constitution/CLAUDE.md` , `constitution/AGENTS.md` , `constitution/QWEN.md` , `constitution/GEMINI.md`) — these ARE the canonical root per §11.4.35, NOT consumer extensions. The full universal rule set lives in `constitution/Constitution.md` ; the sibling carriers (`CLAUDE.md` / `AGENTS.md` / `QWEN.md`) carry the same content for their respective agents. A consuming project's own repo-root `GEMINI.md` (if any) opens with the inheritance pointer and adds only project-specific rules; THIS file is the canonical carrier and does not.

For agents driven from a consuming project, the project's repo-root carrier SHOULD start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/GEMINI.md

All rules in `constitution/GEMINI.md` (and the `constitution/Constitution.md`
it references) apply unconditionally. The project-specific rules below
extend them.
```

Action-prefix recognition (§ . . — LAYER-)

The block below is the **canonical LAYER-1 action-prefix recognition instruction**, embedded VERBATIM (identical text) into every agent context carrier the constitution maintains. That textual identity across all four carriers IS the LAYER-1 universality: Gemini CLI reads this block via this carrier and applies the prefix expansion itself, so the action-prefix system works on Gemini CLI with zero extra setup.

§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09; arrow form 2026-07-02). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules` ; (3) execute the remainder of the prompt under the expanded instruction. **Five EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash), (5) `ACTION_NAME ---> <rest>` (bare arrow) ≡ `PREFIX::ACTION_NAME ---> <rest>` (namespaced arrow). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x` ≡ `BACKGROUND ---> x` ≡ `DEFAULT::BACKGROUND ---> x` . `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`** , and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar` , YAML `key: value` , URLs) in forms 1/2, the arrow-body separator `" ---> "` (one ASCII space on each side) in form 5, and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A` , re-scan, expand `B` , then the residual is the task); a leading `\` escapes the prefix for the `::` , arrow, and slash forms (`\BACKGROUND :: x` , `\BACKGROUND ---> x` , `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 5 forms) but is NOT registered is NEVER silently

> expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **`BACKGROUND`** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). A second registered action **`REMINDER`** re-surfaces previously-scheduled, CRITICAL, status-UNCERTAIN work: FIRST verify the ACTUAL current status from captured evidence (never assume done or not-done — §11.4.6 no-guessing), THEN act on the delta — report the captured proof if genuinely complete, resume from the exact point if partial, action it NOW if not started, or surface the block per §11.4.66/ §11.4.101 — always producing a status verdict (composes §11.4.6 / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.108 / §11.4.130 / §11.4.147). The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.
>
> 11 4 141

Token-efficiency mandate (§ . .)

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents MUST cut token spend (input AND output) toward **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance**

prefix — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost ~0.1× base input (the dominant cost driver — measured ~170K tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); the operative discipline is to **batch governance edits and keep CLAUDE.md + the constitution byte-stable mid-session** — every governance edit busts the prompt cache (BASELINE.md finding); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350–520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal `11.4.N` token so propagation gates pass) with the canonical full text kept gate-scanned in `constitution/Constitution.md` and reachable in one hop — a de-duplication realising §11.4.35, never a deletion; (4) **CodeGraph/retrieval-first over full-file loading** (§11.4.78/§11.4.79/§11.4.80); (5) **output-token reduction** — terse status + `effort:"low"` on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions**. **Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative `usage` object (input/cache_read/cache_creation/output split; NEVER `tiktoken`, NEVER the client-side cost estimate), reproduced N times (§11.4.50), pass = AFTER ≤ 40% of BEFORE OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run MUST show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (`cache_read_input_tokens > 0`) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the *measured* reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes §11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-141-PROPAGATION` (literal `11.4.141`) + recommended gate `CM-TOKEN-EFFICIENCY` + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses →

measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.141. Non-compliance is a release blocker. No escape hatch — no `--skip-token-efficiency`, `--no-cache-governance`, `--assert-reduction-without-measuring`, `--tier-down-reasoning`, `--inline-all-governance`, `--tiktoken-estimate-OK` flag.

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: "ALL changes we do MUST pass through the code review step!!! ALWAYS!!!" EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125's "after all fixes/changes/implementations are done" phrasing could leave open — no "just a doc edit", no "just a one-liner", no "the author already self-reviewed (§11.4.92)", no "trivial change" carve-out. **Independence is load-bearing** — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92's multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp "looks good" is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal

(§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate `CM-COVENANT-114-142-PROPAGATION` (literal `11.4.142`) + recommended gate `CM-EVERY-CHANGE-REVIEWED` (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → `CM-EVERY-CHANGE-REVIEWED` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.142. Non-compliance is a release blocker. No escape hatch — no `--skip-review`, `--no-review`, `--trivial-change-exempt`, `--doc-edit-exempt`, `--self-review-suffices`, `--review-after-commit` flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: "All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that's it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + a consuming project's extensions." Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app's OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app's own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — "the show I picked actually plays" — unproven (the operator's "we just eventually play on 2nd display sample, and that's it mostly" gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app's own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target)

via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title's captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app's accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is `operator_attended` SKIP-with-reason per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): "the routing fired so the title is playing" is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it. Composes §11.4.48 / §11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate `CM-COVENANT-114-143-PROPAGATION` (literal `11.4.143`) + recommended gate `CM-VIDEO-REAL-JOURNEY-TEST` (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPs-with-reason) + paired §1.1 mutation (replace a real-journey test's browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.143. Non-compliance is a release blocker. No escape hatch — no `--sample-playback-ok`, `--skip-real-journey`, `--deep-link-suffices`, `--routing-only-pass`, `--no-title-selection` flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no

resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project's ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project's recovery path already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35. Propagation gate `CM-COVENANT-114-144-PROPAGATION` (literal `11.4.144`) + recommended gate `CM-DEVICE-AVAILABILITY-FOLLOWED` + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.144. Non-compliance is a release blocker. No escape hatch — no `--skip-availability-following`, `--silent-recording-gap-OK`, `--no-reconnect-wait`, `--invent-reconnect-timing`, `--skip-recovery-escalation`, `--autonomous-power-cycle-OK` flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial "refute-the-change" stance to defeat the documented LLM confirmation-bias failure mode) MUST research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec AND the connected features' contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review yet failed at RUNTIME ONLY (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 SOURCE→ARTIFACT→RUNTIME→USER-VISIBLE gap caught only after the fact; §11.4.145 shifts that discovery LEFT. Each angle MUST name the TOOL(S) used + obtain the required DEPTH (the diff, the full changed unit, its declared contract, the spec section, every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/§11.4.69 ("looks fine" forbidden, §11.4.1); a genuinely-N/A angle is recorded `NOT_APPLICABLE: <reason>` per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/§12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-145-PROPAGATION` (literal `11.4.145`) + recommended gate `CM-`

`IMPACT-RESEARCH-PER-CHANGE` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → `CM-IMPACT-RESEARCH-PER-CHANGE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.145. Non-compliance is a release blocker. No escape hatch — no `--skip-impact-research`, `--single-angle-suffices`, `--author-researches-own-change`, `--diff-skim-OK`, `--no-go-overridable`, `--trivial-change-no-research` flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator's exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped `RED_MODE=1→0`) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never "no other issues found"); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per

§11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised cases stated as honest gaps, never silently implied clean. Composes §11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-146-PROPAGATION` (literal `11.4.146`) + recommended gate `CM-REPRODUCE-FIRST-THEN-EXTEND` (every closed defect's fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its `RED_MODE=0` GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce→confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce→confirm pair and no extend-case-set → `CM-REPRODUCE-FIRST-THEN-EXTEND` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.146. Non-compliance is a release blocker. No escape hatch — no `--skip-reproduce-first`, `--fix-without-red`, `--skip-extend-to-all-cases`, `--reproduce-confirm-suffices`, `--defer-extend-to-release-sweep`, `--single-guard-suffices`, `--no-defect-characterisation` flag.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10). Verbatim operator intent: "any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!" Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (`API Error: Server is temporarily limiting requests` rate-limit killed 5 at once, `API Error: socket connection closed unexpectedly` killed 2, one left PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO

mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — "it probably finished before dying" is a forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch timestamp, status from the CLOSED SET `{dispatched | in-flight | crashed | respawned | complete}`), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; "an entry with no dispatch-event cannot show `complete`"); the registry is the SINGLE SOURCE OF TRUTH for "is this work owed?" and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime's own retries are exhausted) flips the entry to `crashed` + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches `complete`; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never blocks the loop for a human), transient causes use the project's ALREADY-DEFINED backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean-restart** — the crashed agent's uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation/ `// always pass` / `_mutated_*` residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent `git worktree` isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) "crash ≠ done" COMPLETION criterion** — a unit is DONE only when an agent reaches `complete` with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is

`dispatched` / `in-flight` / `crashed` / `respawned` -not-yet- `complete`, the zero-idle survey (§11.4.94) treats every non-`complete` entry as an OPEN item to reclaim, and a registry showing `complete` without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40 retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect→wait/backoff→re-attach/respawn→escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-147-PROPAGATION` (literal `11.4.147`) + recommended gate `CM-CRASHED-AGENT-RESPAWN-TRACKED` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a `crashed` entry as the loop's done-condition `complete` without landed evidence, OR drop a dispatched agent without a registry entry → `CM-CRASHED-AGENT-RESPAWN-TRACKED` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.147. Non-compliance is a release blocker. No escape hatch — no `--skip-agent-registry`, `--crash-equals-done`, `--no-respawn`, `--discard-partial-state`, `--blind-commit-partial`, `--forget-dead-agent`, `--loop-done-with-crashed-entries` flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT / §11.4.106 docs_chain) into ONE integrity contract spanning DB ↔ docs ↔ external tracker, adding the operator's three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB `description` column AND the docs AND the tracker, never a stub/§11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every

`Operator-blocked` item (incl. `Blocked` / `BLOCKED` documented alias normalised to canonical `Operator-blocked`, never a silent fork §11.4.6) MUST enumerate the closed list of decisions/actions that would unblock it (`[A]…·[B]…·[C]…` , mirroring §11.4.66's 2–4-option shape); a BLOCKED item with no enumerated choices FAILs; **(D4) regular never-missed bidirectional DB↔docs↔tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (`db-to-md` / `md-to-db` byte-identical round-trip §11.4.93 + tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key `[<ID>]` prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side `created=N updated=M failed=0` proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items + agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes §11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-148-PROPAGATION` (literal `11.4.148`) + recommended gates `CM-ITEM-INTEGRITY-STATUS-TYPE-ID` / `CM-ITEM-COMPREHENSIVE-DESCRIPTION` / `CM-BLOCKED-UNBLOCK-CHOICES` / `CM-TRACKER-SYNC-IDEMPOTENT` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.148. Non-compliance is a release blocker. No escape hatch — no `--item-without-status` , `--item-without-type` , `--item-without-id` , `--stub-description-OK` , `--blocked-without-choices` , `--skip-tracker-sync` , `--one-way-sync-OK` , `--non-idempotent-tracker-push` , `--hardcode-assignee` flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 `item_history` (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles): the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a `test_diary` table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC `date_time` + `tested_by` (closed set `User|Operator|AI-agent|HelixQA`) + `result` (§11.4.45 `PASS|FAIL|SKIP` + detail) + in-depth `observations` (long markdown, facts or explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` §11.4.6, the background a future fix needs) + `action_taken` + `status_changed` +from/to (did this run change status + WHY) + §11.4.69 `evidence_path` + `feature_class`, with a SCHEMA CONSTRAINT making a `PASS` row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item `Diary` / `Diary_Summary` exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 docs_chain-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK model — the item task's description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type `task`) with a `{TODO|In-progress|Completed}` lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; **(e)** MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the `observations` prose is authored by whoever ran the test; tooling only stores/renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run's verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary

doc layout, export formats, tracker sub-task field map, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-149-PROPAGATION` (literal `11.4.149`) + recommended gates `CM-TEST-DIARY-SYNC` / `CM-DIARY-PASS-REQUIRES-EVIDENCE` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.149. Non-compliance is a release blocker. No escape hatch — no `--skip-testing-diary`, `--diary-without-evidence`, `--no-diary-summary`, `--diary-without-tracker-subtask`, `--llm-in-diary-data-path`, `--diary-export-optional` flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: "For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don't have a more serious problem we're unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!" For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation, API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted `Won't-fix: structurally-impossible` (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a "we can't" became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/structural WITHOUT a documented deep-research pass** — no item may be marked `Fixed` / `Implemented` / `Completed` (§11.4.33) OR classified

`structurally-impossible` won't-fix (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes §11.4.8's pass UNCONDITIONAL (every item, however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; "we found nothing worse" requires the enumerated search (§11.4.118). **(D) Latest-source + cited** — LATEST authoritative versions per §11.4.99 (never training-data/ memory), each cited by URL + access date in the research artefact AND the closure commit footer (`Deep-research <date>: <urls>` OR the literal `NO external solution found — original work` per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed↔Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and "we probably don't have a bigger problem" without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate `CM-COVENANT-114-150-PROPAGATION` (literal `11.4.150`) + recommended gate `CM-DEEP-RESEARCH-PER-ISSUE` (every closed/structural-verdicted item carries a

documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation (strip the literal → propagation gate FAILs; close an item or reach a `structural` verdict with no documented multi-angle research artefact / cited footer → `CM-DEEP-RESEARCH-PER-ISSUE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.150. Non-compliance is a release blocker. No escape hatch — no `--skip-deep-research`, `--single-source-suffices`, `--trivial-change-no-research`, `--close-without-research`, `--structural-without-research`, `--serialise-research`, `--research-from-memory-OK` flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: "Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project's release prefix, e.g. `myproject-1.0.0-dev-0.0.1`. The prefix MUST come from `HELIX_RELEASE_PREFIX` in our `.env` if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository." Every release tag AND every version name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project's release prefix, form `<PREFIX>-<version>` (canonical example: `myproject-1.0.0-dev-0.0.1`), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1) `HELIX_RELEASE_PREFIX` from the project's `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked `.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo `<PREFIX>-1.2.0` while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (`<PREFIX>-…-0.0.1` → `<PREFIX>-…-0.0.2` → …), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a

release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var per §11.4.35. Propagation gate `CM-COVENANT-114-151-PROPAGATION` (literal `11.4.151`) + recommended gate `CM-RELEASE-PREFIX-NAMING` (every release tag/version on the main repo + every owned submodule carries the resolved `<PREFIX>-` prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → `CM-RELEASE-PREFIX-NAMING` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.151. Non-compliance is a release blocker. No escape hatch — no `--no-release-prefix`, `--unprefixed-tag`, `--prefix-optional`, `--differing-submodule-prefix` flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13). Verbatim operator intent: "For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!" Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green). STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory, neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL four surfaces** — fatal crashes, ANRs, performance traces/regressions, AND non-

fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), "acknowledged in console" is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: `RED_MODE=1` captures the recorded defect on the pre-fix artifact, `RED_MODE=0` standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4/.1/.6 — "no new issues" requires the enumerated monitored-surfaces+window §11.4.118; "fixed" requires the RED→GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is "closed" only when its guard is GREEN on a clean artifact, never when the console mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is a consuming project's §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a non-fatal with triage context). Composes §11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/

§1.1. Propagation gate `CM-COVENANT-114-152-PROPAGATION` (literal `11.4.152`) + recommended gate `CM-CRASHLYTICS-ISSUE-FULLY-COVERED` (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.152. Non-compliance is a release blocker. No escape hatch — no `--skip-crashlytics-monitoring`, `--monitor-crashes-only`, `--skip-non-fatals`, `--resolve-without-regression-test`, `--console-mark-is-fixed`, `--monitor-once`, `--mute-without-rationale` flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15). Every project MUST maintain, under `docs/features/`, a comprehensive feature Status document set (`Status.md` + its §11.4.56 `Status_Summary.md` companion) enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component→Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer §11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible "confirmed" claim backed by a recorded real-use video of a genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mocked/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35); a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146

retest → re-record → clean GO per §11.4.134 before "confirmed" (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60. (8) **MP4 format REQUIRED.** All video confirmations MUST be in `.mp4` format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). `.cast` files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row's §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146/§1.1. Propagation gate `CM-COVENANT-114-153-PROPAGATION` (literal `11.4.153`) + recommended gates `CM-FEATURE-STATUS-COMPLETE` + `CM-FEATURE-STATUS-VIDEO-CONFIRMED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.153. Non-compliance is a release blocker. No escape hatch — no `--skip-feature-status`, `--feature-without-video`, `--frozen-video-OK`, `--bluff-response-video-OK`, `--skip-video-analysis`, `--feature-ledger-incomplete-OK`, `--no-docx-export`, `--allow-feature-ledger-drift` flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: "recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!" + "All old recording files MUST BE removed when new one starts!" Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never

a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): "remove old" = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed `docs/qa/<run-id>/` evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. Propagation gate `CM-COVENANT-114-154-PROPAGATION` (literal `11.4.154`) + recommended gate `CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.154. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-capture-OK`, `--skip-window-scope`, `--keep-stale-recordings`, `--no-corpus-rotation`, `--full-desktop-recording` flag. **(C) MP4 auto-conversion REQUIRED.** Any `.cast` file produced MUST be auto-converted to `.mp4` via `agg` + `ffmpeg` immediately after capture. The `.mp4` is the primary evidence; `.cast` is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: "All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution's own project-name resolution." Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed `docs/qa/<run-id>/` trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) `HELIX_RELEASE_PREFIX` from `.env` (authoritative, git-ignored §11.4.30, documented in tracked `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so `ls '<PREFIX>---'*` enumerates the corpus; canonical form `<PREFIX>---<feature-or-scope>---<run-id>.<ext>`

(`---` delimits the prefix unambiguously); MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation (rotation removes the agent's OWN `<PREFIX>---*` only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1. Propagation gate `CM-COVENANT-114-155-PROPAGATION` (literal `11.4.155`) + recommended gate `CM-RECORDING-PROJECT-NAME-PREFIX` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix`, `--recording-without-project-name`, `--unprefixed-recording`, `--prefix-optional-for-recording`, `--differing-recording-prefix` flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: "Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!" Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any `on: push` / `schedule` / `workflow_dispatch`). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a `…disabled-local-only` non-`.yml` name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active `.github/workflows/*.yml|yaml` / `.gitlab-ci.yml` / `.gitlab/**` / equivalent at the ROOT of any governed repo/submodule (the only place a provider executes); **(B)** "disabled" = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 `.disabled` / `.disabled-local-only`); a live-`on:` + `if:false` workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs below the root (AOSP `external/**`, `prebuilts/**`, vendored submodules) are INERT (a provider

never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = "does a push to OUR upstream trigger a run?" yes⇒disable, inert⇒document+leave (§11.4.6 verify-not-assume); **(D)** no new CI may be added (release blocker); **(E)** pre-push verify `git ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-ci\.yml$'` empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection required checks, provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-156-PROPAGATION` (literal `11.4.156`) + recommended gate `CM-NO-ACTIVE-CI` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root `.github/workflows/x.yml` to an authored repo → `CM-NO-ACTIVE-CI` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.156. Non-compliance is a release blocker. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator mandate: "Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!" Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4 propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal `11.4.N` anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects' own

CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md "in sync" while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-157-PROPAGATION` (literal `11.4.157`, GEMINI.md INCLUDED) + recommended gate `CM-GEMINI-MD-LOCKSTEP` + paired §1.1 meta-test mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.157. Non-compliance is a release blocker. No escape hatch — no `--skip-gemini-md`, `--gemini-optional`, `--gemini-lag-OK`, `--four-carrier-suffices` flag.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts→real responses→real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→§11.4.153(4)/§11.4.4 fix→retest→re-record), never a confirmation. The testing System MUST ACTUALLY READ every shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — "a video was produced" is NOT evidence, "the System read the screen + confirmed a genuine result" is. HelixQA (§11.4.27) MUST drive this exercise→record→read→score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = `$HOME/Downloads` (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. **Vision analysis MANDATORY for every recording** — after every recording, the agent MUST read the terminal output / video content and verify the feature actually works. A recording without a vision-confirmed verdict is a §11.4.158 violation. Composes §11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-158-PROPAGATION` (literal `11.4.158`) + recommended gates

`CM-INTENSIVE-RECORDING-COVERAGE` + `CM-RECORDING-CONTENT-READ-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.158. Non-compliance is a release blocker. No escape hatch — no `--skip-recording-coverage`, `--video-without-content-read`, `--happy-path-recording-suffices`, `--recording-path-anywhere`, `--unread-recording-OK`, `--skip-helixqa-read` flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen; use macOS `screencapture -l<window_id>`, Linux `xdotool` + `ffmpeg`, or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** — `.mp4` (H.264, `movflags +faststart`, `pix_fmt yuv420p`); `.cast` files are supplementary only; auto-convert via `agg` + `ffmpeg` per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from `HELIX_RELEASE_PREFIX` in `.env` (§11.4.151) or lowercased project root dir name (§11.4.29); format `<project_name>-<feature>-<timestamp>.mp4`; unprefixed = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no "TODO implement" / "simulate" / "for now" patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via `osascript` / `xdotool`); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix→retest→re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent's own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY — not duration-based** — the value of a recording is

NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just "something ran"), LLM responses are REAL content (not simulated, not placeholder, not empty), every claim of "working" is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected recordings** — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent MUST investigate WHY before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output DURING recording (not after); this catches issues immediately and allows corrective action before the recording completes. Classification: universal (§11.4.17). Composes §11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1. Propagation gate `CM-COVENANT-114-159-PROPAGATION` (literal `11.4.159`) + recommended gate `CM-WINDOW-VIDEO-VALIDATED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.159. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-ok`, `--cast-only`, `--skip-vision-validation`, `--no-cleanup`, `--simulated-recording-ok`, `--unprefixed-recording` flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen

content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); capture frames at ≤5s intervals, run OCR/vision with self-validated analyzer (§11.4.107(10)), compare text against patterns, produce per-frame PASS/FAIL with evidence path, surface failures immediately for re-record (§11.4.159(L)). Project-configured parameters per §11.4.35. Honest boundary: does NOT replace §11.4.5 quality analysis nor §11.4.108 runtime-signature; FLAG_SECURE uses §11.4.117 proxy oracle. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-160-PROPAGATION` + recommended gate `CM-VISION-VERIFIED-RECORDING-BRIDGE` + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21). Compact summary: every project MUST use Podman rootless (or equivalent) for ALL containerized workloads — rootful Docker/sudo FORBIDDEN unless §11.4.112 platform constraint documented; `vasic-digital/containers` Submodule (§11.4.76) sole orchestration layer via `pkg/boot` / `pkg/compose` / `pkg/health`; extend upstream per §11.4.74 for missing capabilities; integration tests boot infra on-demand. Honest boundary: does NOT replace §11.4.10 credentials nor §12.3 hygiene. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-161-PROPAGATION` + recommended gate `CM-ROOTLESS-CONTAINER-RUNTIME` + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project with user-facing interfaces MUST use OpenDesign (https://github.com/nexu-io/open-design) as mandatory UI design system — NOT ad-hoc CSS; install as dependency, use token system for all color (light+dark from brand assets), typography, spacing, component tokens; extend upstream (§11.4.74) for unsupported patterns; every UI component ships light+dark + token-diff regression tests via visual regression; no element overlap/font collision/ label overlay. Honest boundary: does NOT replace §11.4.27 functional testing nor §11.4.48/.49 UI testing. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-162-PROPAGATION` + recommended gate `CM-OPENDESIGN-UI-SYSTEM` + paired §1.1 mutation.

§11.4.163 — Universal Media Validation (User mandate). Every recorded artifact MUST pass MEDIA VALIDATION pipeline — OCR/transcription/text parsing, compare vs SPECIFY-phase patterns (§11.4.159(J)), self-validated analyzer

(§11.4.107(10)), structured verdict, pinpoint on FAIL, REAL-TIME trigger option. Paired §1.1 mutation on golden-bad fixture. Classification: universal. Propagation gate `CM-COVENANT-114-163-PROPAGATION` + paired §1.1 mutation.

§11.4.164 — Universal Auto-Propagation Hook (User mandate). Every constitutional pull triggers `post_update_hook.sh` — detects changed files, registers skills/MCP/hooks/scripts, logs summary. Consumer invokes after every pull. Classification: universal. Propagation gate `CM-COVENANT-114-164-PROPAGATION` + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent (User mandate). Every code/media/docs/config output passes independent verifier (§11.4.70/.20), zero-finding GO (§11.4.134). CODE: review+build+mutations+runtime-signature. MEDIA: pipeline+genuine+format. DOCS: exports+revision. CONFIG: syntax+schema+leaks. Self-validated §1.1 mutation. Classification: universal. Propagation gate `CM-COVENANT-114-165-PROPAGATION` + paired §1.1 mutation.

§11.4.166 — REPEALED (operator decision, 2026-06-22). The Universal Semgrep static-analysis mandate is repealed — Semgrep is NO LONGER mandatory. Static analysis remains encouraged, not mandated. Scaffolding, `submodules/semgrep`, MCP/pre-commit/PATH wiring, docs_chain `semgrep_status` context, and the `CM-COVENANT-114-166-PROPAGATION` / `CM-SEMGREP-WIRED` gates removed. Anchor 11.4.166 retired. See constitution `Constitution.md` §11.4.166.

§11.4.167 — Big-work-item feature work-stream lifecycle (User mandate, 2026-06-23). Every BIG work item (new feature OR drastic/large fix) MUST be its own isolated feature work-stream — own sibling project copy, own `feature/<slug>` branch, own per-feature flashable builds + feature tags, SEPARATE from trunk until operator manually approves after testing, trunk merged INTO every stream regularly. (A) one big item = one sibling project copy (`<project>_<slug>`); small changes stay on trunk. (B) disk-feasible CoW/reflink clone MANDATORY (`cp -a --reflink=auto` on btrfs/XFS/ZFS/APFS, else `git worktree`); naive deep copies FORBIDDEN; per-stream `out/` + shared ccache; ~0-disk verified (§11.4.69) before relied on (§11.4.6). (C) own branch + greppable tags (§11.4.151 `<base>-feat-<slug>`), NEVER merged to trunk until §11.4.40 GREEN on clean target → §11.4.41 merge-first → §11.4.113 ff-only. (D) trunk merged INTO every live stream per trunk-tag/daily (merge, never rebase). (E) feature branch + tag cascades to EVERY touched submodule (§11.4.113). (F) single-builder build

QUEUE (§11.4.58/.12.7/.12.8) + finite-device queue (§11.4.119); idle `out/` GC'd (§11.4.77). (G) every change crosses full gauntlet (§11.4.142/.125/.134/.145 + anti-bluff). (H) stream registry (§11.4.116/.147) + atomic `.partial` →rename + §11.4.84 resume + §9.2 retire-backup; bounded §12/.12.6. Honest boundary (§11.4.6): isolation+disk-feasibility+greppable-identity+merge-discipline, NOT correctness (still §11.4.108/.40). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-167-PROPAGATION` (literal `11.4.167`) + gates `CM-FEATURE-WORKSTREAM-COW-CLONE` / `-NO-MERGE-UNTIL-APPROVED` / `-TRUNK-SYNC-CADENCE` / `-SUBMODULE-CASCADE` + paired §1.1 mutations. **Canonical authority:** constitution `Constitution.md` §11.4.167.

§11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate, 2026-06-23). Every generated/exported document (HTML/PDF/DOCX/any §11.4.65 export-scope format) MUST pass INDEPENDENT validation by a reviewer structurally separate from the generator (§11.4.70/.142, NEVER author self-check), on BOTH source AND every exported artifact, ALWAYS, across THREE layers: (1) CONTENT — export faithfully carries source intent+data, nothing dropped/truncated/garbled; (2) TEXTUAL — human-readable, NO raw markup/diagram-source (Mermaid `gantt` / `graph` / `flowchart` / `sequenceDiagram` etc.)/unrendered code-fence leaking as body text (the exact 2026-06-23 raw-gantt-in-PDF failure); (3) FULL VISUAL — diagrams render as IMAGES not source, layout intact, no overlap/cut-off/garble, verified by RENDERING the export (`pdftotext` catches raw source + `pdfimages` confirms rendered images + `pdftoppm` →OCR confirms human-readable content) with captured evidence §11.4.5/.69/.107. Anti-bluff: rubber-stamp ≠ validation; analyzer self-validated golden-good/golden-bad §11.4.107(10) (golden-bad seeded with raw `gantt` MUST FAIL); iterate to clean GO §11.4.134. Forensic FACT (2026-06-23): a "green" Mermaid fix shipped raw gantt source as readable text into user PDFs because validation grepped HTML for `class="mermaid"` and NEVER checked the exported PDF — operator caught it (§11.4.138). Honest boundary (§11.4.6): confirms the exported artifact is faithful/readable/visually-rendered — does NOT prove source content is correct (§11.4.142/.135) nor replace the §11.4.65 mtime freshness gate (this is the READABILITY/FIDELITY layer the mtime gate cannot see); FLAG_SECURE/un-rasterisable surfaces document the gap §11.4.112 + §11.4.117 proxy, never a faked PASS. Classification: universal (§11.4.17). Composes §11.4.65/.73/.107/.117/.135/.138/.142/.159/.163/.165/§1.1. Propagation

gate `CM-COVENANT-114-168-PROPAGATION` (literal `11.4.168`) + recommended gate `CM-EXPORTED-DOC-VISUALLY-VALIDATED` + paired §1.1 mutation. **Canonical authority:** constitution `Constitution.md` §11.4.168.

§11.4.169 — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, 2026-06-25). Every project MUST cover a CLOSED enumerated test-type set, each to as-close-to-100% as the domain permits, every PASS citing rock-solid captured PHYSICAL evidence (§11.4.5/.69/.107), zero false results/zero bluff (§11.4/.1/.6): (1) unit (only layer mocks allowed §11.4.27(A)); (2) integration (real System, infra via containers submodule §11.4.76); (3) e2e; (4) full-automation (autonomous, re-runnable, no manual step §11.4.25/.52/.98, deterministic §11.4.50); (5) Challenges (challenges submodule banks §11.4.27(B)); (6) HelixQA (helix_qa submodule — written test banks/suites per app/service/platform + comprehensive fully-autonomous QA sessions §11.4.27); (7) DDoS/load-flood (clean refuse or graceful degrade); (8) security (authz/injection/secret-leak §11.4.10/crypto/CVE/default-deny + security submodule); (9) stress+chaos (load+contention+boundary+failure-injection, categorised recovery §11.4.85); (10) concurrency/atomicity (no lost updates, transactional atomicity, idempotent replay); (11) race-condition/deadlock (race detector + lock-order analysis; no blocking op in shared-lock region §1); (12) memory (leak census + peak-RSS ceiling + 24h/N-iter soak, no unbounded growth); (13) benchmarking/performance (p50/p95/p99 + throughput vs recorded baseline). The ONLY permitted absence is an honest §11.4.3 SKIP-with-reason, never a silent gap; four-layer §11.4.4(b) enforcement + coverage ledger (§11.4.25/.52); missing required type or OPERATOR_ATTENDED_ONLY = release blocker. Strict enumerated expansion of §11.4.27. Composes §11.4.4/.5/.6/.25/.27/.50/.52/.69/.76/.85/.98/.107/.123/.135/.146/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-169-PROPAGATION` (literal `11.4.169`) + recommended gate `CM-MANDATORY-TEST-TYPES-COVERED` + paired §1.1 mutation. **Canonical authority:** constitution `Constitution.md` §11.4.169.

§11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate, 2026-06-25). Compact summary: every change to ANY user-facing UI surface (screen/component/view/widget/layout/theme on any platform — Android-Compose, iOS-SwiftUI/UIKit, web, desktop, TUI) MUST be proven by DEVICE-INDEPENDENT host-side RENDERED PIXELS before it may be claimed correct — the real component rendered to a PNG ON THE HOST (NO device, NO

emulator, NO running app) via a host-render harness (Compose→Roborazzi/Paparazzi on the JVM; web→Playwright/Storybook snapshot; SwiftUI→snapshot-testing; equivalent per stack), for EVERY relevant screen×state×{light,dark} theme, with the PNG validated by BOTH (i) golden image-diff (per-pixel/perceptual PASS/FAIL) AND (ii) an OCR/vision oracle reading rendered text+labels+control bounds to assert legibility + NO overlap / NO label-over-label / NO clipping / NO off-screen control / NO collapsed-or-giant-unbounded widget (the §11.4.162 layout invariants PROVEN on real pixels). Forensic FACT (2026-06-25): UI work was "validated" by VALUE-EQUALITY unit tests (a hex colour string / an sp/dp size integer / a design-token field == a constant) that NEVER RENDERED A PIXEL — GREEN while the operator opened a broken unbounded giant-button screen. A value/token-equality / property-assertion UI test is FORBIDDEN as the PROOF a UI is correct (it may SUPPLEMENT, NEVER SUBSTITUTE the rendered-pixel proof); a "green" UI suite with no rendered-pixel artefact for the changed surface is a §11.4 PASS-bluff at the visual layer. Self-validated golden-good/golden-bad analyzer §11.4.107(10), thresholds calibrated on the project's own fixtures §11.4.6. "device offline" is NEVER a valid skip — host-render IS the device-independent path; honest §11.4.3 SKIP only where the platform genuinely has no host-render harness (tracked migration item, never a value-only fallback). Honest boundary §11.4.6: host-render proves the COMPONENT renders correctly device-independently per commit — it does NOT prove the running app on real hardware (that remains §11.4.153/.158/.159/.160's live-device recording + read-the-screen layer, which §11.4.170 COMPLEMENTS not replaces — both mandatory), does NOT replace §11.4.162 OpenDesign token/theme governance (§11.4.170 PROVES the rendered RESULT of those tokens), DISTINCT from §11.4.168 exported-document visual validation. Classification: universal (§11.4.17). Composes §11.4.3/.5/.27/.40/.69/.107/.108/.117/.153/.158/.159/.160/.162/.163/.168/.169/§1.1. Propagation gate `CM-COVENANT-114-170-PROPAGATION` (literal `11.4.170`) + recommended gate `CM-HOST-RENDERED-UI-VISUAL-PROOF` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.170. Non-compliance is a release blocker. No escape hatch — no `--value-assertion-proves-ui`, `--skip-rendered-pixel-proof`, `--token-equality-suffices`, `--device-offline-skip-visual`, `--single-theme-only`, `--unvalidated-image-diff-OK`, `--no-ocr-layout-oracle` flag.

When in doubt

- Read `constitution/Constitution.md` for the canonical text of every rule.
- Cross-reference the sibling carriers (`CLAUDE.md` / `AGENTS.md` / `QWEN.md`) — they carry the same universal content for their respective agents.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff (§11.4.6).

§11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29). Every workable item (ATM-NNN ticket) in the project's workable-items database, Issues.md, Fixed.md, Issues_Summary.md, Fixed_Summary.md, and ALL derived documents MUST carry a comprehensive human-readable description that explains the item in plain, non-technical language suitable for non-developers (project managers, stakeholders, team leads, business partners). The description MUST be 5-7 sentences minimum, written so that ANY person — regardless of technical background — can understand: (a) WHAT the item is (the feature, fix, or task in simple terms); (b) WHY it matters (the user/business value); (c) HOW it works (the mechanism, without code references); (d) WHO benefits (the end user or stakeholder); (e) WHAT the expected outcome is (the measurable result). The description is MANDATORY — an item without a human-readable description is a §11.4 violation at the documentation layer. Descriptions MUST NOT use jargon, acronyms without expansion, code references, or technical implementation details as the primary explanation. Technical details MAY appear as supplementary context AFTER the plain-language explanation. The `description` column in the SQLite database (`docs/workable_items.db`) is the SINGLE SOURCE OF TRUTH for item descriptions; all derived documents (Issues.md, Fixed.md, summaries, exports) MUST carry the SAME description text. Pre-build gate `CM-HUMAN-READABLE-DESCRIPTION` (when implemented) walks every item in the DB and asserts the `description` field exists, is non-empty, and is ≥ 50 characters. Paired §1.1 meta-test mutation strips a description → gate FAILs. Classification: universal (§11.4.17). Composes §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (column alignment), §11.4.91 (summary clarity), §11.4.93 (SQLite SSoT), §11.4.148 (item integrity), §11.4.65 (universal export). Propagation gate `CM-COVENANT-114-171-PROPAGATION` (literal `11.4.171`) + paired §1.1 mutation.

§11.4.172 — Mandatory production-readiness planning with realistic timeline projection (User mandate, 2026-06-29). Every project under this Constitution MUST maintain a living production-readiness planning document that: (a) provides REALISTIC timeline projections for all product milestones (MVP, beta, production-ready) based on actual development velocity data (items completed per week, reopening rate, blocking dependencies); (b) identifies ALL danger zones, weaknesses, and risk areas with mitigation strategies; (c) accounts for HW delivery timelines, licensing delays, and external dependency risks; (d) defines the critical path and identifies which items can slip without affecting MVP; (e) includes workstation/build-capacity analysis and optimization recommendations; (f) is updated at least monthly or whenever the item count changes by ≥ 10%; (g) is cross-referenced against the workable-items database for accuracy. The planning document MUST live at `docs/research/production_planning_<YYYYMMDD>/ANALYSIS.md` with HTML+PDF exports per §11.4.65. Honest boundary (§11.4.6): projections are estimates based on measured velocity — they guarantee the methodology is sound, NOT that specific dates will be met. Classification: universal (§11.4.17). Composes §11.4.6 (no-guessing — projections based on data, not wishful thinking), §11.4.40 (full retest before tag), §11.4.42 (iteration discipline), §11.4.93 (SQLite SSoT), §11.4.108 (four-layer verification). Propagation gate `CM-COVENANT-114-172-PROPAGATION` (literal `11.4.172`) + paired §1.1 mutation.

§11.4.173 — Containerized + distributed build mandate (User mandate, 2026-06-29). EVERY build of EVERY component (source compile, artifact/package/installer/container-image production, codegen, asset render — for ANY language/platform: Go, Android/Gradle, desktop, web, native, firmware) MUST run INSIDE a specialized build container provisioned via the `digital.vasic.containers` submodule (§11.4.76) — NEVER on the bare host. The build containers MUST be DISTRIBUTED to the designated remote build host(s) (e.g. `thinker.local`) via the SAME containers-submodule distribution mechanism the infra uses (§11.4.76 Distributor / remote compose over SSH, §11.4.161 rootless), so the build EXECUTES on the remote build host (offloading the developer/main host); once the build completes the produced artifacts MUST be brought BACK to the originating main host (scp/rsync/volume copy) for use/flashing/distribution. Building outside a container, or on the bare host, is FORBIDDEN — a release blocker (the "works on my machine" / unreproducible-build class §11.4.76 exists to prevent). The build-host target + build-container definitions are config-injected (§11.4.28, never hardcoded in the submodule); a missing build-container capability is added by

EXTENDING the containers submodule upstream (§11.4.74), never by an ad-hoc host build. Honest boundary (§11.4.6): the containerized+distributed build guarantees reproducibility + host-isolation + capacity offload — it does NOT replace the §11.4.40 full-suite retest, §11.4.108 four-layer artifact→runtime verification, or §11.4.38 installable-asset evidence (those still run against the brought-back artifact). Classification: universal (§11.4.17). Composes §11.4.76 (containers submodule — sole orchestration layer), §11.4.161 (rootless runtime), §11.4.74 (extend-don't-reimplement), §11.4.28 (config injection / decoupling), §11.4.24 (build-resource stats), §11.4.82 (iteration speedup — persistent caches in the container), §11.4.121 (no-commit-while-build-writes-artifacts), §11.4.38 (installable-asset evidence on the brought-back artifact), §11.4.108 (artifact→runtime verification), §12.6 (host memory ceiling — offloading the build preserves the main host). Propagation gate `CM-COVENANT-114-173-PROPAGATION` (literal `11.4.173`) + recommended gate `CM-CONTAINERIZED-DISTRIBUTED-BUILD` (every build runs via the containers submodule on the remote build host; a bare-host build is detected + FAILs) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.173. Non-compliance is a release blocker. No escape hatch — no `--build-on-host`, `--skip-container-build`, `--local-build-ok`, `--no-distributed-build`, `--bare-host-build` flag.

§11.4.174 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29). On any shared host where other projects/users/agents may run concurrently, every inspection of or action on a process, build, daemon, port, lock, or system-state signal MUST first positively VERIFY the target is OURS before diagnosing or acting. Attribute a process to us ONLY by a strong discriminator — cwd inside this project's tree, argv naming our scripts/artifacts/repo path, a PID we launched + recorded (§11.4.147), or a lock/port path under our tree — NEVER a loose `pgrep java/gradle/node` name match (it matches every project on the host). NEVER kill/signal/restart a process not positively ours (other projects' gradle/java/podman/zig are off-limits — operator-workload-safety, §12/§11.4.133); the §12.8 daemon-kill remediation MUST be scoped to OUR daemons. When our work waits on host contention from another project's process, surface it (§11.4.66/§11.4.101) — block-don't-break. Every pgrep/ps filter MUST exclude self-matches + other-project matches. Forensic anchor (FACT 2026-06-29): a 200%-CPU java+gradlew on the shared host was a separate `catalogizer` project's gradle test (cwd=Projects/catalogizer), nearly mis-attributed to our build. Composes §11.4.6/§11.4.66/§11.4.101/§11.4.147/§12/§12.8/§11.4.133. Classification: universal

(§11.4.17). Propagation gate `CM-COVENANT-114-174-PROPAGATION` (literal `11.4.174`) + recommended gate `CM-PROCESS-OWNERSHIP-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.174. Non-compliance is a release blocker. No escape hatch — no `--assume-process-ours`, `--loose-pgrep-ok`, `--kill-any-matching-process`, `--skip-ownership-check` flag.

§11.4.176 — Conflict-free multi-track work-division + exactly-once claim registry + capability-aware deadlock-proof device-lock (User mandate, 2026-07-02). Two mandatory multi-track arbitration layers, conflict-free by construction: (A) exactly-once work-item/logical-group claim registry (extends §11.4.147/§11.4.116; disjoint file-scope §11.4.58 L3 + worktree/CoW §11.4.167; no codebase loss §9.2/§11.4.113/§11.4.84/§11.4.41); (B) capability-aware deadlock-proof device-lock (§11.4.119 single-owner; all-or-nothing + non-blocking + TTL-reap breaks all 4 Coffman conditions; append-only JSONL + atomic snapshot §11.4.116); (C) universal/decoupled (§11.4.17/§11.4.28/§11.4.35) + evidence-based resource-tuning (§11.4.24/§11.4.6, never guessed). Classification: universal (§11.4.17). Gates CM-WORK-DIVISION-EXCLUSIVE-CLAIM + CM-DEVICE-LOCK-DEADLOCK-FREE + CM-COVENANT-114-176-PROPAGATION + paired §1.1 mutation. Composes §11.4.6 §11.4.17 §11.4.24 §11.4.28 §11.4.35 §11.4.41 §11.4.58 §11.4.84 §11.4.85 §11.4.103 §11.4.113 §11.4.116 §11.4.119 §11.4.147 §11.4.167 §12.6 §9.2 §11.4.176.

§12.11 — Maximal dynamic resource utilization for containerized builds (User mandate, 2026-07-03). The containerized build path (its OWN cgroup + own MemoryMax → OOM-kills ITSELF, never user.slice) MUST use the maximal SAFE fraction of host capacity, computed DYNAMICALLY per host: auto-detect nproc/MemTotal/RLIMIT_NPROC (never hardcode, §11.4.6), reserve explicit host headroom (cores+RAM), scale -j against BOTH the memory model AND the process rlimit (privileged nproc-raise for full -j; EAGAIN-safe -j otherwise), never break/bottleneck/wedge. NO fixed low -j for the containerized path. The §12.6 60% ceiling + bare-metal -j cap REMAIN, SCOPED to user.slice-resident work — §12.11 does NOT weaken §12.6, it scopes it. Classification: universal (§11.4.17). Gate CM-MAXRES-DYNAMIC-BUILD + CM-COVENANT-12-11-PROPAGATION + paired §1.1 mutation. Composes §12.1 §12.2 §12.3 §12.6 §12.10 §11.4.6 §11.4.24 §11.4.133 §9.2 §11.4.35 §12.11.

§11.4.177 — Developer-tooling project-decoupling + invocation-directory operation (User mandate, 2026-07-04). Compact summary: no project-specific script / hook / alias / binary may be wired (copied / symlinked / PATH-exported / aliased) into a global or shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the INVOCATION directory (cwd / explicit path / config arg), NEVER a hardcoded project path; a project-specific helper lives in + runs from its own repo, a genuinely-reusable helper is promoted to the shared toolkit ONLY after being stripped of every project-specific assumption (paths, device serials, package names, region endpoints) + taking its target from the invocation context; a project script reachable on the global/shared PATH is a decoupling violation of equal severity to importing project-specific code into a shared submodule (§11.4.28); forensic FACT 2026-07-04 — a project cwd-hook symlinked into the global Claude-Toolkit PATH re-coupled every alias to one hardcoded checkout; honest boundary §11.4.6 — decoupling guarantees project-agnostic, not correct (still needs the tool's own tests). Classification: universal (§11.4.17). Composes §11.4.28/.29/.35/.11/.6. Propagation gate `CM-COVENANT-114-177-PROPAGATION` (literal `11.4.177`) + recommended gate `CM-TOOLING-PROJECT-DECOUPLED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.177. Non-compliance is a release blocker. No escape hatch — no `--global-symlink-ok`, `--hardcode-project-path`, `--wire-into-shared-path` flag.

§11.4.178 — Track-qualified identity for parallel work streams (session-name collision ban) (User mandate, 2026-07-04). Compact summary: parallel work streams sharing hardware / a git backing store / a tmux-session namespace / a lock namespace / any single-owner resource MUST be addressed by a TRACK-QUALIFIED identity (`<project>__<track>__<role>`), NEVER a bare directory basename; multiple checkouts/clones/worktrees sharing basename `atmosphere` collapse into one session/lock/log key + cross-wire tmux targets, lock files, device claims, log dirs (observed collision 2026-07-04); every registry row / session name / lock path / log dir / device claim for a ≥2-concurrent-claimant resource MUST carry the track-qualified key, a bare-basename identity for such a resource is a violation; honest boundary §11.4.6 — a unique identity prevents cross-wiring, it does not itself arbitrate ownership (that is §11.4.119 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor)). Classification: universal (§11.4.17). Composes §11.4.111/.119/.29/.58 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor). Propagation

gate `CM-COVENANT-114-178-PROPAGATION` (literal `11.4.178`) + recommended gate `CM-TRACK-QUALIFIED-IDENTITY` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.178. Non-compliance is a release blocker. No escape hatch — no `--bare-basename-ok`, `--skip-track-qualifier`, `--shared-session-name` flag.

§11.4.179 — Corruption-isolated parallel git streams (own-.git independent clones, not shared-common-dir worktrees) (User mandate, 2026-07-04). Compact summary: when corruption-isolation is a requirement, parallel git work streams MUST each be an isolated repo with its OWN `.git` (own object store, own index, own lock namespace), NOT `git worktree` checkouts sharing one common `.git`; a shared common-dir is a single point of failure — one stale `index.lock` / `.commit_all.lock` / `.push_all.lock` freezes EVERY stream at once (observed 9-h all-track freeze 2026-07-04) + one corrupted object store loses every stream; each stream's `git rev-parse --git-common-dir` MUST resolve INSIDE its own tree, a common-dir resolving to a shared parent is a violation; where CoW/reflink disk-feasibility is the constraint use the §11.4.167 reflink-clone-with-own-`.git` path (btrfs/XFS/ZFS/APFS reflink shares extents on disk while keeping a per-stream `.git`, so isolation + disk-feasibility both hold); honest boundary §11.4.6 — own-`.git` contains lock/corruption blast radius, it does NOT remove per-repo stale-lock reaping (§11.4.180) nor ff-only no-force integration (§11.4.113). Classification: universal (§11.4.17). Composes §11.4.167/.119/.58/§9.2 + **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor). Propagation gate `CM-COVENANT-114-179-PROPAGATION` (literal `11.4.179`) + recommended gate `CM-GIT-STREAMS-OWN-COMMON-DIR` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.179. Non-compliance is a release blocker. No escape hatch — no `--shared-common-git`, `--worktree-when-isolation-required`, `--skip-git-isolation` flag.

§11.4.180 — Commit/push (single-writer) wrappers MUST auto-reap provably-stale locks before acquiring (User mandate, 2026-07-04). Compact summary: every commit / push / single-writer wrapper MUST, before acquiring its own lock, auto-reap any PROVABLY-stale git lock — one whose recorded holder PID is DEAD (`kill -0` fails), OR (no PID recorded) older than a defined threshold AND with no live wrapper/git process holding that repo; the reap covers the wrapper's own lock plus the git-internal existence-based locks (`index.lock`, `HEAD.lock`,

`refs/**/*.lock`) git does NOT auto-release on holder death; it MUST NEVER remove a lock whose holder is ALIVE (removing a live-held lock corrupts a concurrent writer — §9.2); each reap decision (REAPED / KEPT + reason) is logged as captured evidence (§11.4.6 — liveness PROVEN via `kill -0`, never assumed); a wrapper that blocks indefinitely on a dead-holder lock (observed 9-h freeze 2026-07-04) is a violation; honest boundary §11.4.6 — reaping clears dead-holder deadlock, it does NOT substitute for the own-`.git` isolation of §11.4.179. Classification: universal (§11.4.17). Composes §11.4.84/.88/.6/§9.2/§11.4.116/§11.4.179. Propagation gate `CM-COVENANT-114-180-PROPAGATION` (literal `11.4.180`) + recommended gate `CM-WRAPPER-STALE-LOCK-AUTO-REAP` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.180. Non-compliance is a release blocker. No escape hatch — no `--skip-stale-lock-reap`, `--force-remove-live-lock`, `--block-on-dead-holder` flag.

§11.4.181 — Consistent controlled feature-branch naming: one feature/logic-group ⇒ exactly ONE canonical branch name across the main repo + all owned submodules (User mandate, 2026-07-05). Compact summary: one feature / logical group of workable items MUST map to EXACTLY ONE canonical branch name used IDENTICALLY on the main repo AND every touched owned submodule — NEVER two differently-named branches for one feature/group, NEVER per-repo naming drift; (1) single canonical name per group minted ONCE per the §11.4.167 D scheme (`feature/<slug>` branch + `<prefix>-<base>-feat-<slug>` tag, `<slug>` lowercase snake/kebab §11.4.29), used identically across the main repo + every branched submodule (an untouched submodule stays on its base branch); (2) single source of truth — the canonical name is recorded ONCE in the §11.4.176 exactly-once claim registry / the §11.4.93 workable-items `logic_group → branch` mapping and LOOKED UP, never re-invented (creating a branch whose name ≠ the group's registered canonical name, or minting a 2nd name for a group that already has one, is a §11.4.6 no-guessing violation); (3) mechanical enforcement (§11.4.75) — recommended gate `CM-BRANCH-NAME-CONSISTENCY` FAILs on an unregistered feature-branch name / two divergent names for one group / a submodule branch name diverging from the group's main-repo canonical name, paired §1.1 mutation (register one group under two divergent names → gate FAILs); (4) risk-free reconciliation (§9.2/§11.4.113/§11.4.84) — a mis-named/duplicate branch is reconciled by MERGING its commits onto the canonical branch (union, no commit lost, no `-s ours`/reset), ff-only push

to all upstreams (NEVER force-push), retiring the mis-named branch only after the merge is confirmed on all mirrors. Classification: universal (§11.4.17). Composes §11.4.167/.176/.178/.179/.28/.29/.93/.113/.84/.6/.75/§9.2. Propagation gate `CM-COVENANT-114-181-PROPAGATION` (literal `11.4.181`) + recommended gate `CM-BRANCH-NAME-CONSISTENCY` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.181. Non-compliance is a release blocker. No escape hatch — no `--allow-divergent-branch-names`, `--skip-branch-registry`, `--rename-branch-freely`, `--per-repo-branch-name-OK`, `--two-names-one-feature-OK` flag.

§11.4.182 — Track+branch work-stream identity label `(T<N>/<branch>)` on every agent/subagent label + every operator-facing work-stream reference (User mandate, 2026-07-05). Compact summary: EVERY agent / subagent / work-stream label AND every operator-facing reference to a work stream MUST START with a `(T<N>/<branch>)` prefix identifying the track + git branch (e.g. `(T1/main) ATM-312 MPV drm_prime investigation`, `(T2/feature/mistiq-vader) SPK-XXX ...`) so parallel-track work (`/mnt/track1..4`, each on its own branch) is never ambiguous; (1) universal label form on every agent-dispatch description, status report, resume/handoff doc, progress line; (2) DETERMINISTIC derivation never guessed (§11.4.6) — `<N>` from the checkout path `/mnt/track<N>/…` (honest `?` off-track, never a guessed number), `<branch>` from `git rev-parse --abbrev-ref HEAD` (reference labeler `scripts/multitrack/track_branch_label.sh`); (3) mechanical enforcement (§11.4.75 / §11.4.109-class) — a PreToolUse guard hook (`constitution/scripts/hooks/guard-track-branch-label.sh`, inherited BY REFERENCE per §11.4.177, never copied) BLOCKS any Agent/Task/TaskCreate dispatch whose `description` (or `subagent` label) does not START with `^\(T[0-9]+/[^)]+\)`, non-agent tools untouched; pre-build gate `CM-TRACK-BRANCH-LABEL` (labeler+hook exist/exec/parseable, hook wired in settings, convention doc exists) + propagation gate `CM-COVENANT-114-182-PROPAGATION` (literal `11.4.182`); (4) honest boundary (§11.4.6) — a `?` field is a valid honest label, never a bluff, and the enforced numeric-track format surfaces (blocks) an off-track dispatch rather than mislabeling it. Classification: universal (§11.4.17). Composes §11.4.178 (track-qualified identity — the label-prefix instantiation applied to every agent/subagent label + operator-facing reference) / §11.4.29 / §11.4.75 / §11.4.109 / §11.4.177 / §11.4.6 / §11.4.58 / §11.4.103 / §11.4.119 / §11.4.116 / §11.4.147. Propagation gate `CM-COVENANT-114-182-PROPAGATION` (literal

`11.4.182`) + recommended gate `CM-TRACK-BRANCH-LABEL` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.182. Non-compliance is a release blocker. No escape hatch — no `--skip-track-label`, `--unlabeled-agent-OK`, `--bare-work-stream-reference`, `--no-track-branch-prefix`, `--label-from-memory` flag.

§11.4.183 — Maximal multi-agent utilization per work-stream + full-constitution-application + zero-bluff mandate (User mandate, 2026-07-07). Compact summary: every track / work-stream MUST maximize multi-agent working approaches — subagent-driven development (§11.4.20/§11.4.70), independent review agents (§11.4.125/§11.4.142/§11.4.165/§11.4.134), parallel background streams with auto-backfill (§11.4.58/§11.4.103), and every other applicable agent form — so every track produces the MOST completed work at the HIGHEST quality in the LEAST time; every track MUST apply the ENTIRE constitution (every rule, mandatory constraint, guide, and derived technology — NOTHING ignored, skipped, avoided, or deemed less-relevant); there MUST NEVER be false / faulty / untested / unverified results or bluff of any kind anywhere (§11.4 anti-bluff covenant, captured evidence §11.4.5/§11.4.69/§11.4.107); honest boundary §11.4.6 — "maximize agents" is bounded by §12.6 60% memory + §11.4.58 parallel-agent cap + genuine parallelizability + §11.4.119 single-resource-owner, spawning contending/redundant agents is NOT maximization, the bar is maximum USEFUL parallel throughput not agent count. Classification: universal (§11.4.17). Composes §11.4.20/.58/.70/.92/.94/.97/.103/.119/.125/.126/.134/.142/.165/§12.6. Propagation gate `CM-COVENANT-114-183-PROPAGATION` (literal `11.4.183`) + recommended gate `CM-MAX-AGENTS-PER-TRACK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.183. Non-compliance is a release blocker. No escape hatch — no `--single-agent-OK`, `--skip-review-agents`, `--partial-constitution-application`, `--serialise-parallelisable-work` flag.

§11.4.184 — Mandatory SonarQube static-analysis CLI + local tooling installed and PATH-discoverable (User mandate, 2026-07-06). Compact summary: every project/host that runs static analysis MUST have the SonarQube scanner CLI (`sonar-scanner`) installed AND durably discoverable on the system PATH via the shell rc (`.bashrc` / `.zshrc`) — exactly like the Firebase/GitHub/GitLab CLIs — PLUS the shared `constitution/scripts/sonarqube/` tooling (`sonarqube_install_check.sh` / `sonarqube_lib.sh` / `sonarqube_run_scan.sh` / `sonarqube_container.sh` + `compose/`) consumed

BY REFERENCE (§11.4.28 decoupled + §11.4.177 project-agnostic + §11.4.80-style inherited, NEVER copied per project); (1) installed + PATH-durable — the scanner resolves in every fresh shell via a durable rc export, not an ephemeral session-only PATH; (2) shared tooling present + GREEN — `sonarqube_install_check.sh` exits 0 (scanner + rootless podman §11.4.161 + podman-compose + curl + ES-ready `vm.max_map_count`) before any scan-dependent work; (3) anti-bluff §11.4.6 — "installed/configured" PROVEN by `command -v sonar-scanner` resolving AND install-check exit 0, NEVER assumed, a claimed-installed-but-absent CLI is a §11.4 tooling-layer bluff; (4) rootless §11.4.161 — the local SonarQube server runs via rootless podman, never rootful docker/sudo; honest boundary §11.4.6 — names the SonarQube SCANNER CLI + shared local-server tooling, and per §11.4.166 (former universal Semgrep static-analysis mandate REPEALED) SonarQube is a DISTINCT operator-mandated tool not a Semgrep re-instatement, mandating TOOLING availability not a scan on every build. Classification: universal (§11.4.17). Composes §11.4.28/.36/.74/.75/.109/.161/.166/.177/.6. Propagation gate `CM-COVENANT-114-184-PROPAGATION` (literal `11.4.184`) + recommended gate `CM-SONARQUBE-CLI-INSTALLED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.184. Non-compliance is a release blocker. No escape hatch — no `--skip-sonarqube-install`, `--sonar-cli-optional`, `--ad-hoc-sonar-path`, `--rootful-sonar-OK` flag.

§12.12 — Process/thread-limit (RLIMIT_NPROC) awareness for parallel subagent/multi-process work (User mandate, 2026-07-07). Compact summary: heavy parallel subagent/multi-process work is bounded by the OS per-user process/thread limit `ulimit -u` (`RLIMIT_NPROC`, Linux counts ALL threads+processes of the real UID, not per-process); the ambient tooling fleet (MCP servers, tokio/V8/libuv worker runtimes, `mongod`/`chrome`/`prisma`-class MCP servers, local LLM servers) consumes a large FIXED fraction before project work starts — FORENSIC FACT (2026-07-07): a host at `ulimit -u 4096` sat at ~3900-4005 threads from the ambient fleet alone (<150 headroom); a 4th Go-heavy subagent (`GOMAXPROCS=nproc` spawns dozens of threads) hit `EAGAIN` / `failed to create new OS thread (errno=11)` and crashed tooling; stopping 3 non-essential MCP servers freed ~1345 threads (3827→2482). Mandate: BEFORE scaling parallel subagents/processes, check thread headroom (`ulimit -u` soft + `ulimit -Hu` hard + live `ps -L --no-headers -u "$USER" | wc -l`) and treat exhaustion as a §12 host-safety event that YIELDS UNCONDITIONALLY — the

ORTHOGONAL second exhaustion axis alongside §12.6's memory ceiling (abundant RAM ≠ abundant thread budget); signals `EAGAIN` / `fork: retry` / `failed to create new OS thread` / `cannot allocate memory` on fork/clone (distinct from OOM, §11.4.6 no-guessing — never conflate); low headroom ⇒ SERIALIZE, avoid `GOMAXPROCS=nproc`-class spikes, never dispatch blind (a subagent hitting `EAGAIN` mid-`git push` is a §9.2 data-safety risk). Three raising techniques: (a) no-restart `prlimit --pid <PID> --nproc=<soft>:<hard>` (root, e.g. `su -c`) — new children of a running process inherit the raise; (b) persistent `/etc/security/limits.conf` (`<user> hard/soft nproc <N>`) or systemd `LimitNPROC=<N>`, next login; (c) self-raise `ulimit -u <N>` up to the existing hard ceiling within the current session — `su -c "ulimit -u N"` sets the limit ONLY inside that transient subshell, NOT the already-running session. Freeing threads (stopping non-essential MCP servers) REQUIRES operator authorization (§11.4.122, no autonomous tooling removal); CAUTION `pkill -f` / `pgrep -f` match the killer's OWN command line — always exclude the caller's `$$` / `$PPID` (and conductor PID) from any cmdline-pattern kill list to avoid self-kill. Classification: universal (§11.4.17). Composes §12/.6/.7/§11.4.58/.101/.103/.122/§9.2. Propagation gate `CM-COVENANT-12-12-PROPAGATION` (literal `12.12`) + recommended gate `CM-NPROC-HEADROOM-CHECK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §12.12. Non-compliance is a release blocker. No escape hatch — no `--ignore-thread-limit`, `--dispatch-blind`, `--skip-headroom-check`, `--autonomous-tooling-kill`, `--assume-ulimit-scope` flag.

§11.4.185 — Manual QA-team testing as the mandatory FINAL confirmation of done (User mandate, 2026-07-07). Compact summary: no set of features / scope of work / release may be considered fully completed until it has received MANUAL TESTING confirmation by the project's QA team as the FINAL confirmation step — everywhere and always, never forgotten; every automated gate (§11.4 anti-bluff covenant, §11.4.5/§11.4.69 captured evidence, §11.4.52 autonomous validation, §11.4.40 full-suite retest, §11.4.108 four-layer verification, §11.4.132 risk-ordered validation) is NECESSARY but NOT SUFFICIENT — manual QA is the FINAL human sufficiency gate; ADDS to (never weakens) §11.4.52's mandatory autonomous-validation-path requirement — a feature needs BOTH the autonomous path AND the manual-QA final confirmation, never one substituting the other; pipeline: autonomous-GREEN → build → flash/deploy → autonomous on-device validation → hand off to QA team → ONLY after QA-team manual confirmation is

the scope fully-completed / tag-eligible; the §11.4.126 autonomous-loop release-tag terminal condition now REQUIRES this confirmation; honest boundary §11.4.6 — the agent MUST hand off + WAIT, never self-certify/simulate/infer the manual step, while continuing every other non-blocked item in parallel per §11.4.87/.94/.97/.101 (the wait parks one scope, not the loop). Classification: universal (§11.4.17). Composes §11.4/.5/.6/.40/.52/.69/.108/.126/.129/.130/.132. Propagation gate `CM-COVENANT-114-185-PROPAGATION` (literal `11.4.185`) + recommended gate `CM-MANUAL-QA-FINAL-CONFIRMATION` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.185. Non-compliance is a release blocker. No escape hatch — no `--skip-manual-qa`, `--automation-suffices`, `--self-certify-manual-step`, `--assume-qa-confirmed`, `--autonomous-only-completion` flag.